

OSN - Mini Project 1

AI-Usage PDF

Vardaan Goel
2025117001

write a Makefile using these flags:
gcc -std=c23 \
-D_POSIX_C_SOURCE=200809L \
-D_XOPEN_SOURCE=700 \
-Wall -Wextra -Werror \
-Wno-unused-parameter \
-fno-asm \
your_file.c

Makefile ⓘ ⓘ

CC = gcc
CFLAGS = -std=c23 -D_POSIX_C_SOURCE=200809L -D_XOPEN_SOURCE=700 -Wall -Wextra -Werror:

SRC_DIR = src
SRCS = \$(wildcard \$(SRC_DIR)/*.c)
OBJS = \$(SRCS:.c=.o)
TARGET = shell.out

all: \$(TARGET)

\$(TARGET): \$(OBJS)
 \$(CC) \$(CFLAGS) -o \$@ \$^

\$(SRC_DIR)/%.o: \$(SRC_DIR)/%.c
 \$(CC) \$(CFLAGS) -c \$< -o \$@

clean:
 rm -f \$(SRC_DIR)/*.o \$(TARGET)

+ Ask Gemini Pro Extended ⓘ

read file line by line parse num, time and path. if path contains the dest substring find score for it score should be from 1 to 0.1 which goes down linearly and becomes 0.1 after 90 days

Here is the exact C code block generated from your prompt:

C



```
while (fgets(line, sizeof(line), f)) {
    HopEntry entry;
    if (sscanf(line, "%d %ld %[^\\n]", &entry.freq, &entry.timestamp, entry.path)

        if (strstr(entry.path, dest) != NULL) {
            double age_seconds = difftime(current_time, entry.timestamp);
            double max_age = 90.0 * 24.0 * 3600.0; // 90 days

            double factor = 1.0 - (age_seconds / max_age);
            if (factor < 0.1) {
                factor = 0.1;
            }

            entry.score = entry.freq * factor;
            matches[match_count++] = entry;
        }
    }
}
fclose(f);
```

+ Ask Gemini

Pro Extended



Gemini is AI and can make mistakes.

whats the way to get current user's name, hostname, and current working directory so i can build a prompt. include handling of errors in case anything fails



To get those details in C, you'll need to use a few standard POSIX system calls. Here is the code block that safely fetches the username, hostname, and working directory, complete with fallback values if any of the system calls fail.

You'll need these headers included first:

C



```
#include <unistd.h>
#include <pwd.h>
#include <limits.h>
#include <string.h>
#include <stdio.h>
```

Here is the logic to populate your variables:

C



```
char host[HOST_NAME_MAX];
char cwd[PATH_MAX];
char *username = NULL;

// 1. Get the Username
struct passwd *pw = getpwuid(getuid());
if (pw) {
```

+ Ask Gemini

Pro Extended ▼



Gemini is AI and can make mistakes.

```

struct passwd *pw = getpwuid(getuid());
if (pw) {
    username = pw->pw_name;
}
if (username == NULL) {
    username = "unknown"; // Fallback if user is not found
}

// 2. Get the Hostname
if (gethostname(host, sizeof(host)) != 0) {
    strncpy(host, "unknown", sizeof(host)); // Fallback if hostname fails
}

// 3. Get the Current Working Directory
if (getcwd(cwd, sizeof(cwd)) == NULL) {
    perror("getcwd failed"); // Print the specific error to stderr
    return; // Exit the prompt function since we can't display the path
}

```

How it works:

- `getuid()` + `getpwuid()` : `getuid` fetches the user ID of the calling process, and `getpwuid` looks up the corresponding password database entry to grab the human-readable username.
- `gethostname()` : Writes the system's hostname into your provided buffer. If it returns anything other than `0`, it failed, so we manually copy "unknown" into the buffer.
- `getcwd()` : Grabs the absolute path of the current directory. If it returns `NULL` (usually due to permissions or the directory being deleted out from under you), it triggers `perror` to print the exact system error.

👍 🗨 📄 ...

+ Ask Gemini

Pro Extended ▾



Gemini is AI and can make mistakes.

write function to get file path for shell's history file and write to given buffer, place file in home directory only failure is home directory being null

Here is a C function that retrieves the home directory using `getenv("HOME")` and constructs the full path to a shell history file (e.g., `~/.shell_history`).

As requested, the only point of failure it checks for is if the home directory environment variable returns `NULL`.

C



```
#include <stdio.h>
#include <stdlib.h>

/**
 * Gets the shell history file path and writes it to the provided buffer.
 *
 * @param buffer      The character array where the path will be written.
 * @param buffer_size The maximum size of the buffer to prevent overflows.
 * @return            0 on success, -1 if the home directory is NULL.
 */
int get_history_path(char *buffer, size_t buffer_size) {
    const char *home_dir = getenv("HOME");

    // The only failure condition: HOME directory is null
    if (home_dir == NULL) {
        return -1;
    }

    // Write the path to the buffer (change ".shell_history" to your specific file name)
    snprintf(buffer, buffer_size, "%s/.shell_history", home_dir);

    return 0;
}
```

+ Ask Gemini

Pro



Gemini is AI and can make mistakes.

in c function, get the metadata for a given path, if it is a file print its info if it is directory, open it and print every file and directory in it, check for l flag and do accordingly

Here is a complete C program that implements this behavior. It uses standard POSIX system calls to check file metadata (`stat` / `lstat`) and traverse directories (`opendir` / `readdir`).

It mimics the behavior of the `ls` command, checking a boolean `l_flag` to decide whether to print just the names or the detailed metadata (permissions, owner, size, and modification time).

The C Implementation

C



```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
#include <dirent.h>
#include <pwd.h>
#include <grp.h>
#include <time.h>
#include <string.h>
#include <stdbool.h>

// Helper function to print file permissions in the "-rw-r--r--" format
void print_permissions(mode_t mode) {
    printf((S_ISDIR(mode)) ? "d" : (S_ISLNK(mode)) ? "l" : "-");
    printf((mode & S_IRUSR) ? "r" : "-");
    printf((mode & S_IWUSR) ? "w" : "-");
    printf((mode & S_IXUSR) ? "x" : "-");
    printf((mode & S_IRGRP) ? "r" : "-");
```

+ Ask Gemini

Pro



Gemini is AI and can make mistakes.

```

        printf((mode & S_IXUSR) ? "x" : "-");
        printf((mode & S_IRGRP) ? "r" : "-");
        printf((mode & S_IWGRP) ? "w" : "-");
        printf((mode & S_IXGRP) ? "x" : "-");
        printf((mode & S_IROTH) ? "r" : "-");
        printf((mode & S_IWOTH) ? "w" : "-");
        printf((mode & S_IXOTH) ? "x" : "-");
    }

    // Prints the information for a single file or directory entry
    void print_file_info(const char *name, const struct stat *sb, bool l_flag) {
        if (!l_flag) {
            printf("%s\n", name);
            return;
        }

        // Long format (-l) requires gathering user, group, and time strings
        print_permissions(sb->st_mode);

        struct passwd *pw = getpwuid(sb->st_uid);
        struct group *gr = getgrgid(sb->st_gid);

        char time_buf[256];
        strftime(time_buf, sizeof(time_buf), "%b %d %H:%M", localtime(&sb->st_mtime));

        printf(" %2lu %-8s %-8s %8ld %s %s\n",
            (unsigned long)sb->st_nlink,
            pw ? pw->pw_name : "unknown",
            gr ? gr->gr_name : "unknown",
            (long)sb->st_size,
            time_buf,
            name);
    }

    // Core function to process the given path
    void process_path(const char *path, bool l_flag) {
        struct stat sb;

```

+ Ask Gemini

Pro



Gemini is AI and can make mistakes.

```

C
void process_path(const char *path, bool l_flag) {
    struct stat sb;

    // Get metadata for the target path (lstat doesn't follow the symlink itself)
    if (lstat(path, &sb) == -1) {
        perror("Error reading path");
        return;
    }

    // Check if it is a directory
    if (S_ISDIR(sb.st_mode)) {
        DIR *dir = opendir(path);
        if (!dir) {
            perror("Error opening directory");
            return;
        }

        struct dirent *entry;
        while ((entry = readdir(dir)) != NULL) {
            // Skip the current (.) and parent (..) directory pointers
            if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..") == 0)
                continue;
        }

        // Construct the full path so stat() can find the file
        char full_path[1024];
        snprintf(full_path, sizeof(full_path), "%s/%s", path, entry->d_name);

        struct stat entry_stat;
        if (lstat(full_path, &entry_stat) == -1) {
            continue; // Skip if we can't read the metadata
        }

        print_file_info(entry->d_name, &entry_stat, l_flag);
    }
    closedir(dir);
} else {
    // It's a standard file or symlink
    // ... (code for file processing) ...
}

```

+ Ask Gemini

Pro



Gemini is AI and can make mistakes.

write the function to print file's lines in reverse order using lseek, reading file in small chunks

Here is the complete `print_rev_file` function.

Reading a file backwards in chunks is tricky because lines can be split across two different chunks. To solve this, the algorithm uses a dynamically allocated `tail` buffer to carry incomplete lines from the end of one chunk into the beginning of the next.

C



```
static void print_rev_file(FileInfo *f, int n_flag) {
    int fd = open(f->name, O_RDONLY);
    if (fd == -1) {
        perror("peek");
        return;
    }

    off_t pos = lseek(fd, 0, SEEK_END);
    char chunk[4096];

    // 'tail' stores the piece of a line that got cut off by the chunk boundary
    char *tail = NULL;
    size_t tail_size = 0;

    int is_first = 1;
    int current_num = f->start_num;
```

+ Ask Gemini

Pro Extended



Gemini is AI and can make mistakes.

```

// Loop backwards until we hit the start of the file (pos == 0)
while (pos > 0) {
    // 1. Calculate how much to read (a full 4096 bytes, or whatever is left)
    off_t to_read = (pos > (off_t)sizeof(chunk)) ? (off_t)sizeof(chunk) : pos;
    pos -= to_read;

    // 2. Move the read head and grab the chunk
    lseek(fd, pos, SEEK_SET);
    read(fd, chunk, to_read);

    int end = to_read - 1;

    // 3. Ignore the very last newline in the file so we don't print a phantom b'
    if (is_first && chunk[end] == '\n') {
        end--;
    }
    is_first = 0;

    // 4. Scan backwards through the chunk looking for newlines
    for (int j = end; j >= 0; j--) {
        if (chunk[j] == '\n') {
            size_t p_len = end - j;
            const char *part = chunk + j + 1;

            // Check if the reconstructed line will be completely empty
            int empty = is_empty(part, p_len) && (tail_size == 0 || is_empty(tail));

            if (n_flag && !empty) {
                printf("%d ", current_num--);
            }

            // Print the text from this chunk, followed immediately by the leftover
            if (p_len > 0) fwrite(part, 1, p_len, stdout);
        }
    }
}

```

+ Ask Gemini

Pro Extended



Gemini is AI and can make mistakes.

```

        // Print the text from this chunk, followed immediately by the leftover
        if (p_len > 0) fwrite(part, 1, p_len, stdout);
        if (tail_size > 0) fwrite(tail, 1, tail_size, stdout);
        printf("\n");

        // We finished printing this line, so wipe the tail clean
        free(tail);
        tail = NULL;
        tail_size = 0;

        end = j - 1; // Move the end marker to just before this newline
    }
}

// 5. If we hit the start of the chunk but haven't found a newline yet,
// we save the remaining text into the tail for the next loop iteration.
if (end >= 0) {
    size_t p_len = end + 1;
    char *new_tail = malloc(p_len + tail_size);

    // Note: because we are reading backwards, the new chunk data comes BEFORE
    memcpy(new_tail, chunk, p_len);
    if (tail_size > 0) {
        memcpy(new_tail + p_len, tail, tail_size);
    }

    free(tail);
    tail = new_tail;
    tail_size += p_len;
}
}

```

// 6. After the loop, if there is anything left in the tail, add it to the new chunk

+ Ask Gemini

Pro Extended



Gemini is AI and can make mistakes.

```

        if (end >= 0) {
            size_t p_len = end + 1;
            char *new_tail = malloc(p_len + tail_size);

            // Note: because we are reading backwards, the new chunk data comes BEFORE
            memcpy(new_tail, chunk, p_len);
            if (tail_size > 0) {
                memcpy(new_tail + p_len, tail, tail_size);
            }

            free(tail);
            tail = new_tail;
            tail_size += p_len;
        }
    }

    // 6. After the loop, if there is anything left in the tail, it's the very first
    if (tail_size > 0) {
        if (n_flag && !is_empty(tail, tail_size)) {
            printf("%d ", current_num--);
        }
        fwrite(tail, 1, tail_size, stdout);
        printf("\n");
        free(tail);
    }

    close(fd);
}

```

👍 💬 ↻ 📄 ...

+ Ask Gemini

Pro Extended ▼



Gemini is AI and can make mistakes.

function to read all lines from stdin into memory and store them array of strings strip all\n and \r

Here is the complete `buffer_stdin` function that uses a dynamically resizing array to safely load all of standard input into memory, while stripping out the newline and carriage return characters:

C



```
static void buffer_stdin(FileInfo *f) {
    // Start with a reasonable capacity and allocate memory
    int cap = 128;
    f->lines = malloc(cap * sizeof(char*));
    f->total_lines = 0;
    f->valid_lines = 0;

    char *line = NULL;
    size_t len = 0;
    ssize_t read_bytes;

    // Read line by line until EOF (Ctrl+D)
    while ((read_bytes = getline(&line, &len, stdin)) != -1) {
        // If our array is full, double its capacity
        if (f->total_lines >= cap) {
            cap *= 2;
            f->lines = realloc(f->lines, cap * sizeof(char*));
        }

        // Strip trailing \n (newline)
        if (read_bytes > 0 && line[read_bytes - 1] == '\n') {
            line[read_bytes - 1] = '\0';
        }
    }
}
```

+ Ask Gemini

Pro Extended



Gemini is AI and can make mistakes.

```

    f->total_lines = 0;
    f->valid_lines = 0;

    char *line = NULL;
    size_t len = 0;
    ssize_t read_bytes;

    // Read line by line until EOF (Ctrl+D)
    while ((read_bytes = getline(&line, &len, stdin)) != -1) {
        // If our array is full, double its capacity
        if (f->total_lines >= cap) {
            cap *= 2;
            f->lines = realloc(f->lines, cap * sizeof(char*));
        }

        // Strip trailing \n (newline)
        if (read_bytes > 0 && line[read_bytes - 1] == '\n') {
            line[read_bytes - 1] = '\0';
        }

        // Strip trailing \r (carriage return, common in Windows files)
        if (read_bytes > 1 && line[read_bytes - 2] == '\r') {
            line[read_bytes - 2] = '\0';
        }

        // Duplicate the cleaned string and store it in our array
        f->lines[f->total_lines++] = strdup(line);

        // Check if the line has visible text for line numbering (-n)
        if (!is_empty(line, strlen(line))) {
            f->valid_lines++;
        }
    }

    // getline allocates memory for the buffer, so we must free it
    free(line);
}

```

Answer now

+ Ask Gemini

Pro Extended ▾



Gemini is AI and can make mistakes.

c function that returns 1 if path is valid file and it is executable and 0 otherwise

Here is the C function to do this. It uses `stat()` to ensure the path is a regular file (ignoring directories) and `access()` to verify the current user has execute permissions.

C



```
#include <sys/stat.h>
#include <unistd.h>

int is_executable_file(const char *path) {
    struct stat st;

    // stat() retrieves file information and follows symlinks. Returns 0 on success.
    if (stat(path, &st) == 0) {
        // S_ISREG checks if it's a regular file (not a directory or device)
        // access(..., X_OK) checks if the current user has execute permission
        if (S_ISREG(st.st_mode) && access(path, X_OK) == 0) {
            return 1;
        }
    }

    return 0; // Not a file, not executable, or doesn't exist
}
```

+ Ask Gemini

Pro v



Gemini is AI and can make mistakes.

```

C
int is_executable_file(const char *path) {
    struct stat st;

    // stat() retrieves file information and follows symlinks. Returns 0 on success
    if (stat(path, &st) == 0) {
        // S_ISREG checks if it's a regular file (not a directory or device)
        // access(..., X_OK) checks if the current user has execute permission
        if (S_ISREG(st.st_mode) && access(path, X_OK) == 0) {
            return 1;
        }
    }

    return 0; // Not a file, not executable, or doesn't exist
}

```

How it works:

1. `stat(path, &st)` : Fetches the file's metadata. If it returns `-1`, the file doesn't exist or cannot be accessed, so the function safely returns `0`.
2. `S_ISREG(st.st_mode)` : Ensures the path points to a standard file. (Without this, the function might return `1` for a directory, as directories also have "execute" bits to allow entering them).
3. `access(path, X_OK)` : Specifically asks the operating system if the user running the program is allowed to execute the target path.

👍 🗨️ ↻ 📄 ...

+ Ask Gemini

Pro ▾



Gemini is AI and can make mistakes.

write in c a block using fork() to create child processes.
execute programs in the child processes

Here is the standard C code block for forking a process and executing a program.

This uses the classic `fork()` + `execv()` + `waitpid()` pattern, which is the foundational way UNIX systems (and custom shells) run commands!

C



```
// Create a new child process
pid_t pid = fork();

if (pid < 0) {
    // Fork failed (e.g., out of memory or process limits reached)
    perror("fork error");
}
else if (pid == 0) {
    // --- CHILD PROCESS ---
    // execv completely replaces this child process with the target program.
    // It requires the absolute/resolved path, and a NULL-terminated array of arguments
    execv(resolved_path, exec_args);

    // IMPORTANT: execv NEVER returns if it succeeds.
    // If the code reaches this line, it means the execution failed!
    perror("execv error");
    // (never returns)
}
```

+ Ask Gemini

Pro Extended



Gemini is AI and can make mistakes.

```

}
else if (pid == 0) {
    // --- CHILD PROCESS ---
    // execv completely replaces this child process with the target program.
    // It requires the absolute/resolved path, and a NULL-terminated array of arguments.
    execv(resolved_path, exec_args);

    // IMPORTANT: execv NEVER returns if it succeeds.
    // If the code reaches this line, it means the execution failed!
    perror("execv error");
    exit(EXIT_FAILURE);
}
else {
    // --- PARENT PROCESS ---
    // The parent (your shell) waits here until the child process terminates.
    int status;
    waitpid(pid, &status, 0);
}

```

How this works:

1. `fork()` clones your shell. For a split second, there are two identical versions of your shell running at the exact same time.
2. In the child clone (`pid == 0`), `execv()` overwrites the clone's memory with the new program (like `ls` or `echo`) and runs it.
3. In the parent (`pid > 0`), `waitpid()` pauses your shell so it doesn't print the next

+ Ask Gemini

Pro Extended ▼



Gemini is AI and can make mistakes.

c block that sets up feeder for multiple input files, for more than 1 files create pipe and fork a new process inside the child read file descriptors data into a buffer and write them to end of pipe

Here is the C code block that sets up a feeder process to stream data from multiple input files into a pipe:

C



```
int pipefd[2] = {-1, -1};
pid_t feeder_pid = -1;

// If there are multiple input files, we need a feeder process to combine them
if (in_count > 1) {
    if (pipe(pipefd) < 0) {
        perror("pipe error");
    }

    feeder_pid = fork();
    if (feeder_pid == 0) {
        // --- FEEDER CHILD PROCESS ---
        // Close the read end of the pipe since this process only writes to it
        close(pipefd[0]);

        char buf[4096];
```

+ Ask Gemini

Flash-Lite ▾



Gemini is AI and can make mistakes.

```

    if (pipe(pipefd) < 0) {
        perror("pipe error");
    }

    feeder_pid = fork();
    if (feeder_pid == 0) {
        // --- FEEDER CHILD PROCESS ---
        // Close the read end of the pipe since this process only writes to it
        close(pipefd[0]);

        char buf[4096];
        ssize_t bytes_read;

        // Loop through each input file descriptor sequentially
        for (int i = 0; i < in_count; i++) {
            // Read from the file and write its contents into the pipe's write-end
            while ((bytes_read = read(in_fds[i], buf, sizeof(buf))) > 0) {
                write(pipefd[1], buf, bytes_read);
            }
            // Close each file descriptor as we finish reading it
            close(in_fds[i]);
        }

        // Close the write end of the pipe to signal EOF to the main process, then
        close(pipefd[1]);
        exit(EXIT_SUCCESS);
    }
}

```

+ Ask Gemini

Flash-Lite ▾



Gemini is AI and can make mistakes.

c block to fork child process to run command with input redirection. if 1 file, use dup2 to redirect to STDIN_FILENO if more than 1 redirect to stdin then call execl to run command.

Here is the C code block for forking the process and applying the input redirection:

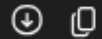
C



```
pid_t pid = fork();
if (pid < 0) {
    perror("fork error");
}
else if (pid == 0) {
    // --- CHILD PROCESS ---
    if (in_count == 1) {
        // Single file: use dup2 to redirect the file descriptor to STDIN
        dup2(in_fds[0], STDIN_FILENO);
    } else if (in_count > 1) {
        // Multiple files: redirect the read-end of the pipe to STDIN
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);
        close(pipefd[1]);
    }

    // Close all raw file descriptors in the child
```

C



```
        dup2(in_fds[0], STDIN_FILENO);
    } else if (in_count > 1) {
        // Multiple files: redirect the read-end of the pipe to STDIN
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);
        close(pipefd[1]);
    }

    // Close all raw file descriptors in the child
    for (int i = 0; i < in_count; i++) close(in_fds[i]);

    // Run the command
    execv(p, args);
    perror("execv error");
    exit(EXIT_FAILURE);
}
else {
    // --- PARENT PROCESS ---
    if (in_count > 1) {
        close(pipefd[0]);
        close(pipefd[1]);
    }

    for (int i = 0; i < in_count; i++) close(in_fds[i]);

    // Wait for child processes to finish
    waitpid(pid, NULL, 0);
    if (feeder_pid > 0) {
        waitpid(feeder_pid, NULL, 0);
    }
}
```

c block to open multiple files for output redirection. check file flags and open file with 0644 permissions

Here is the C code block to handle opening multiple output files, setting the correct flags, applying 0644 permissions, and handling errors cleanly:

C



```
int out_fds[1000];

for (int i = 0; i < out_count; i++) {
    // Determine flags: always Write-Only and Create, then choose Append or Trunc
    int flags = O_WRONLY | O_CREAT | (out_append[i] ? O_APPEND : O_TRUNC);

    // Open with 0644 permissions (rw-r--r--)
    out_fds[i] = open(out_files[i], flags, 0644);

    // If the file fails to open (e.g., permission denied or invalid path)
    if (out_fds[i] < 0) {
        fprintf(stderr, "cshell: unable to create file for writing\n");

        // Cleanup: Close all previously opened input files
        for (int j = 0; j < in_count; j++) close(in_fds[j]);

        // Cleanup: Close all successfully opened output files before this failure
        for (int j = 0; j < i; j++) close(out_fds[j]);
    }
}
```

+ Ask Gemini

Pro Extended ▾



Gemini is AI and can make mistakes.



```
int out_fds[1000];

for (int i = 0; i < out_count; i++) {
    // Determine flags: always Write-Only and Create, then choose Append or Trunc
    int flags = O_WRONLY | O_CREAT | (out_append[i] ? O_APPEND : O_TRUNC);

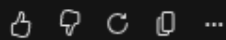
    // Open with 0644 permissions (rw-r--r--)
    out_fds[i] = open(out_files[i], flags, 0644);

    // If the file fails to open (e.g., permission denied or invalid path)
    if (out_fds[i] < 0) {
        fprintf(stderr, "cshell: unable to create file for writing\n");

        // Cleanup: Close all previously opened input files
        for (int j = 0; j < in_count; j++) close(in_fds[j]);

        // Cleanup: Close all successfully opened output files before this failure
        for (int j = 0; j < i; j++) close(out_fds[j]);

        return; // Abort execution completely
    }
}
```



+ Ask Gemini

Pro Extended ▾



Gemini is AI and can make mistakes.

c block to set up a loop for executing pipeline of commands. store the process ids for them and iterate through them get pointer to commands arguments handle errors

Here is the C code block to initialize the pipeline tracking variables and set up the main execution loop, exactly as requested:

C



```
int in_fd = STDIN_FILENO; // Tracks the read-end of the previous pipe
pid_t pids[256];
pid_t feeder_pids[256];
pid_t writer_pids[256];

// --- 2. LOOP THROUGH EACH COMMAND IN THE PIPELINE ---
for (int c = 0; c < num_cmds; c++) {
    int start = pipe_idx[c] + 1;
    int end = pipe_idx[c + 1];
    int cmd_count = end - start;
    char** cmd_input = &input[start];

    // Check for empty commands (e.g., consecutive pipes like `||` or dangling `
    if (cmd_count == 0) {
        fprintf(stderr, "cshell: syntax error near unexpected token `|\n");
        break;
    }
}
```

+ Ask Gemini

Pro Extended ▾



Gemini is AI and can make mistakes.

```

int in_fd = STDIN_FILENO; // Tracks the read-end of the previous pipe
pid_t pids[256];
pid_t feeder_pids[256];
pid_t writer_pids[256];

// --- 2. LOOP THROUGH EACH COMMAND IN THE PIPELINE ---
for (int c = 0; c < num_cmds; c++) {
    int start = pipe_idx[c] + 1;
    int end = pipe_idx[c + 1];
    int cmd_count = end - start;
    char** cmd_input = &input[start];

    // Check for empty commands (e.g., consecutive pipes like `||` or dangling `
    if (cmd_count == 0) {
        fprintf(stderr, "cshell: syntax error near unexpected token `|\n");
        break;
    }

    // Setup the pipe for the NEXT command to read from (if we aren't the last co
    int fd[2] = {-1, -1};
    if (c < num_cmds - 1) {
        if (pipe(fd) < 0) {
            perror("pipe error");
        }
    }
}

```

👍 🗨️ ↻ 📄 ...

+ Ask Gemini

Pro Extended ▼



Gemini is AI and can make mistakes.